Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Ders, doğrusal sıralama ödevinin sunumlarıyla başlar. Ardından fonksiyonlara parametre aktarımında tip dönüşümü (C ile Java karşılaştırması) ve döngülerle yazılmış (iteratif) algoritmaların analizi işlenir: temel işlem (basic operation), iterasyon sayısı, $T(n)$ fonksiyonu ve forward substitution. Dersin sonunda öğrencilerin kod soruları ve uzaktan eğitim üzerine bir sohbet vardır.
- Ödev sunumları
  - Ödevin amacı: geçen dersteki iki doğrusal sıralama algoritmasında karşılaşılan sorunlara (eşit elemanlar ve büyük değerler yüzünden büyük bellek ihtiyacı) çözüm önermek. İlk 7 çalışan çözüme bonus verilecekti. Hocanın beklediği çözüm gelmedi.
  - Bir öğrencinin kodu: her turda hem en küçüğü hem en büyüğü bulup ikisini de yerine koyan bir sıralama. Hoca bunun çift yönlü selection sort olduğunu söyler. Dış döngü $n/2$ kez, iç döngü `i + 1`'den `n - i - 1`'e kadar döner, yani her turda yaklaşık $n - 2i - 1$ adım. Hoca klasik selection sort ile 1, 10, 100 milyon elemanlık rastgele dizilerde (iki diziye aynı değerler kopyalanıp, `printf` olmadan, başta ve sonda zaman ölçülerek) süre karşılaştırması yapılmasını önerir. Her adımda iki karşılaştırma olduğu için farkın başa baş çıkabileceğini de ekler.
  - Hoca, Quick Sort'un hızlı olmasının bir sebebini tek adımda iki elemanın yer değiştirebilmesi olarak anlatır. Quick Sort özyinelemeli algoritmalarla birlikte işlenecektir.
  - Başka bir öğrencinin kodu: yeni diziyi dizinin elemanlarının toplamı kadar açmak. Hoca bunun sorunu çözmediğini gösterir.
    - Toplam, en büyük elemandan daima büyüktür. 1 2 3 4 5 için 5 yerine 15'lik yer açılır. 1 milyon gibi bir değer varsa yine çok büyük dizi gerekir.
    - Negatif elemanlar olursa dizi boyutu negatif olur.
    - `toplam` değişkenine ilk değer verilmeden toplama yapılmış. Bellekte o anda ne varsa ondan devam edilir.
    - `int B[toplam]` gibi değişkene bağlı boyutla dizi tanımlanmış ve dosya `.cpp` olarak kaydedilmiş. Hoca bunun C'de hata olduğunu, büyük değerlerde segmentation fault alınabileceğini söyler.
    - Dizi fonksiyonun ortasında tanımlanmış. C'de değişkenler başta tanımlanmalıdır.
  - Hoca, iç içe olmayan iki ayrı `for` döngüsünün ($2n$) tek döngüden (n) asimptotik olarak farklı olmadığını, ama iç içe iki döngünün n^2'ye çıkabileceğini hatırlatır. Ek bir döngüden kaçınmak için yanlış bir boyut kullanmak iyi bir takas değildir.
  - Bir öğrencinin kodunda bir fonksiyonun parantezi kapanmamış, `j` ilk değersiz kullanılmış ve tanımlanmamış bir değişken var. Kod biçimlendirmesi (girinti, indentation) üzerine kısa bir konuşma geçer.
- Kodlama standartları üzerine
  - Hoca, C yazım kurallarının (değişkenlerin başta tanımlanması, dizi boyutlarının sabit verilmesi, fonksiyonlara değer aktarımı) kendi uydurduğu bir stil olmadığını, bölümdeki diğer hocaların da bunu istediğini söyler. Örneğin global değişken kullanmamanın işletim sistemlerindeki race condition ile ilgili bir gerekçesi vardır.
  - Tutorial siteleri faydalıdır, ama özel bir durum için doğru görünen bilgiler genele uygulandığında yanlış olabilir. Hoca, derin öğrenme üzerine bir firma seminerinde bir saat içinde 5-6 teknik yanlış duyduğunu anlatır.
- Parametre aktarım mekanizmalarının tekrarı
  - Call by value ve call by reference. Call by reference ile değişkenlerin değeri değiş-

tirilebilir ve bir fonksiyon birden fazla değer döndürebilir. `return` en fazla bir değer döndürür.
  - Örnek: `void g(int *t, int *l) { *t = 0; *l = 0; }` ve `g(&x, &y);`. `x` ve `y` sıfırlanır.
  - `scanf("%d %d %c", &x, &y, &c);` de aynı mekanizmayı kullanır: üç değişkenin adresi gider, üçünün değeri değişir.
- Fonksiyon çağrısında tip dönüşümü
  - C bu kontrolü programcıya bırakır. Derleme veya çalışma zamanında hata vermeden dönüşüm yapar. Hoca küçük tipten büyüğe dönüşümde de veri kaybı olabileceğini söyledi; doğrusu asıl kayıp büyükten küçüğe dönüşümde olur. Küçükten büyüğe dönüşümde değer korunur. Bu çok yapılan bir hatadır.
  - Derste denenen fonksiyonlar: `void f(int t) { printf("%d--%c", t, t); }` ve `void h(char t) { ... }` (aynı gövde).
  - `f(3)`: `3--` ve ardından ekranda kalp işareti. `h(3)`: önce `int` olan 3 `char`'a dönüştürülür, sonuç aynıdır.
  - `f('A')`: `65--A`. Karakter sabiti tam sayıya dönüştürülür.
  - Fonksiyona string gönderilmesi (büyük olasılıkla `f("BA")`): string'in bellekteki adresi tam sayıya dönüştürülmeye çalışılır. Derleyici yalnızca uyarı verdi (`makes integer from pointer without a cast`), ekranda anlamsız bir sayı çıktı. Bir diziyi fonksiyona aktarırken ilk elemanın adresinin gittiği hatırlatılır.
  - `f(5.40)`: derste beklenen 5 yerine -1 ve -5 gibi değerler görüldü. Hoca sebebini bilmediğini, ama hata alınmamasının önemli olduğunu söyler.
  - `h(256)` 0, `h(257)` 1 verdi. Derleyici `overflow in implicit constant conversion` uyarısı verdi. Değer 256'ya göre mod alınmış gibi davranır.
  - Java karşılaştırması: Java'da aynı adlı fonksiyon farklı parametrelerle tekrar tanımlanabilir (overloading). C'de bu yoktur. Java, büyük bir tipi küçük bir tipe dönüştürmeye (`possible lossy conversion from long to char`) derleme zamanında izin vermez. Çağrı birden fazla tanıma uyuyorsa `ambiguity` hatası verir.
  - Bu dönem için C'de stringler dışında çok fazla konu kalmadı. Diziler, matrisler, fonksiyonlar ve pointer'larla sayısal analizdeki ve bu dersteki bütün algoritmalar yazılabilir.
- Doğrusal sıralama ödevine hocanın önerdiği çözüm
  - Algoritma bellek-zaman ödünleşimini (trade-off) gösterir. Herkesin kabul edeceği tek bir çözüm yoktur.
  - Eşit elemanlar için: değerleri indis olarak kullanıp her hücrede frekans saklamak. 6 3 8 2 5 9 6 dizisinde 6'nın hücresine 2 yazılır, sonra dizi baştan sona gezilip her değer frekansı kadar yazılır.
  - Bellek için: yalnızca dizide geçen değerleri iki sütunlu (değer, frekans) bir yapıda insertion sort mantığıyla sıralı tutmak. Her yeni değer yerine eklenir, varsa frekansı artırılır. Tablo en fazla eleman sayısı kadar satır olur, en büyük değer kadar yer açmaya gerek kalmaz.
  - Daha iyi çözümler (linked list, hashing) Yapısal Programlama, Veri Yapıları ve Algoritma Analizi derslerinde görülecektir.
- İteratif algoritmaların analizi
  - Fonksiyonlar iki türlüdür: iteratif (yukarıdan aşağıya, döngülerle) ve recursive (fonksiyon kendini çağırır). Bu ders iteratif fonksiyonlar işlenir.
  - Amaç, kodun n'e bağlı çalışma adımı sayısını veren $T(n)$ fonksiyonunu bulmak ve iki algoritmayı $T_A(n)$ ve $T_B(n)$ ile karşılaştırmaktır. Aynı şey bellek için ($M_A(n)$ ve $M_B(n)$) de yapılabilir.
  - Örnek: $n!$ hesabında temel işlem çarpmadır ve yaklaşık n kez yapılır.
  - İki kavram: temel işlem (basic operation) ve iterasyon sayısı (number of iterations).
  - Analizde değişken tanımları, `scanf` ve `printf` gibi fonksiyonlar dikkate alınmaz. Atama, karşılaştırma, artırma gibi operatörler (unary, binary, ternary) sayılır. Bunların makine kodunda karşılıkları vardır (karşılaştırma için `CMP` veya `SUB`). Her birine 1 birimlik maliyet verilir.
  - Ayrıntılı sayım örneği: `T = 0; for (i = 1; i <= N; i++) T = T + 1;`
    - `T = 0` ve `i = 1`: 2 atama.
    - `i <= N`: $N + 1$ karşılaştırma. Son kontrol `i = N + 1` için yapılır ve döngüden çıkılır.
    - `i++`: hocanın sayımında $N + 1$.
    - `T + 1` ve atama: N toplama ve N atama.

    * Toplam yaklaşık $4N+4$. Sabitler ve katsayılar atılıp en büyük dereceli terim alınır: $T(N) \approx N$. $4n^2 + 4n - 3$ çıksaydı n^2 denirdi.
  - Gerçek sürede komutlar birbirinden farklı sürede çalışır. Analizde hepsine 1 birim denir.
  - Temel işlemle sayma: döngüdeki karşılaştırma, artırma veya atamadan biri de seçilebilir. Ama `T`'nin son değeri sorulduğu için temel işlem toplama alınır. Ayrıntılı analizdeki ek terimler gereksiz ayrıntıdır.
  - Forward substitution: değişkenin ilk birkaç iterasyondaki değerleri yazılır ve desen (pattern) bulunur, sonra toplam veya çarpım sembolüyle ifade edilir.
- Derste analiz edilen döngüler (`N = 10` ile çalıştırıldı)
  - T_1: `T = T + 1`. $\sum_{i=1}^{n} 1 = n$. Sonuç 10.
  - T_2: `T = T + 2`. $\sum_{i=1}^{n} 2 = 2n$. Sonuç 20.
  - T_3: `T = T + i`. Değerler 1, 1+2, 1+2+3, ... $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$. Sonuç 55.
  - T_4: `T = T + 2*i`. $2\sum_{i=1}^{n} i = n(n+1)$. Sonuç 110.
  - T_5: `T = T + T`, `T` başta 1. Başta 0 olsaydı hep 0 kalırdı. Değerler 2, 4, 8, 16, ... Her adımda iki katına çıkar, yani çarpım sembolü gibi çalışır: $\prod_{i=1}^{n} 2 = 2^n$. Sonuç 1024.
  - T_6: iç içe döngü, `j` 1'den n^2'ye, `T = T + 1`. $\sum_{i=1}^{n} \sum_{j=1}^{n^2} 1 = n^3$. Sonuç 1000.
  - T_7: aynı döngü, `T = T + i`. İç toplamda `i` sabittir: $\sum_{i=1}^{n} n^2 i = n^2 \frac{n(n+1)}{2} = \frac{n^3(n+1)}{2}$. Sonuç 5500.
  - T_8: aynı döngü, `T = T + j`. İç toplam $\frac{n^2(n^2+1)}{2}$, `i`'ye bağlı değildir: $\frac{n^3(n^2+1)}{2}$. Sonuç 50500.
- Son değer ile karmaşıklık farklı şeylerdir
  - T_3'te `T`'nin son değeri $\frac{n^2+n}{2}$'dir. Ama döngü n kez döner ve her turda 1 toplama yapılır, yani karmaşıklık n'dir.
  - $\frac{n^2+n}{2}$ bir algoritmanın karmaşıklık fonksiyonu olsaydı karmaşıklık n^2 olurdu.
  - Benzer şekilde T_5'in karmaşıklığı n'dir. Ama karmaşıklığı 2^n olan bir algoritma üstel (exponential) zamanlıdır, polinom zamanlı değildir. Örnek: n bitle üretilebilecek bütün şifreleri (2^n tane) denemek. Hoca alt küme problemini de bu sınıfta sayar.
  - T_6, T_7, T_8'in karmaşıklığı aynıdır: dış döngü n, iç döngü n^2 kez döner. Toplam n^3, yani üçüncü dereceden polinom.
- Duyurular
  - Gelecek ders: `for` sınırlarının değiştiği, `if` ve `while` içeren daha karmaşık döngülerin analizi ve bir sıralama algoritmasının (selection sort, bubble sort) analizi. Terim toplam formüllerinin listesi verilecek. Rastgele sayı üretimi için zaman kalmazsa ek ders yapılacak.
- Ders sonrası sorular
  - `int A[100] = {-1};` yalnızca ilk elemanı -1 yapar, kalanlar 0 olur. Bütün elemanları -1 yapmak için döngü gerekir.
  - Count dizisi eleman sayısına (`n`) göre açılıp `count[5555]` gibi değerlere erişilince program çöktü. Hoca bunun dizinin dışına erişim (boundary hatası) olduğunu ve bu algoritmanın düzgün yazılması için dinamik bellek gerektiğini söyler. Şimdilik dizileri büyük bir sabit boyutla (ör. 500) tanımlayın.
  - Aynı değişkeni iki kez tanımlamak ve dizi tanımını kodun ortasına koymak C'de hatadır. Bütün değişkenler her fonksiyonun başında tanımlanmalıdır. Pascal'daki `var` bölümü gibi.
  - `#define` ile tanımlanan adlar değişken değildir. Derlemeden önce önişlemci (preprocessor) tarafından metin olarak yerine konur (substitution).
  - Algoritma ile gerçekleştirme (implementation) arasındaki fark: akış diyagramında doğru olan bir algoritma, dile özgü kurallar bilinmezse koda hatalı aktarılır. Hoca bunu her dilde aynı anlama gelen ama farklı kurulan sıfat cümlelerine benzetir.
  - Matrisi genişletmek (3 × 4'ü 4 × 4 yapmak) bugünkü bilgilerle mümkün değildir. Matris baştan bir boyut büyük tanımlanmalıdır.
  - Gelecek dönem Yapısal Programlama'da kısa bir C tekrarından sonra pointer, dinamik bellek, structure ve dosya işlemleri işlenecektir.

Hocanın Özellikle Vurguladığı Kısımlar

- Kodu `.c` olarak kaydedin. C++ dosyası gelirse dönem sonu projesi doğrudan kabul edilmez.
- Değişkenleri fonksiyonun başında tanımlayın. Dizi boyutunu bir değişkenle vermeyin, sabit veya `#define` kullanın.
- Kümülatif toplam yapılan değişkene mutlaka ilk değer verin.
- C tip dönüşümlerinde hata vermez. Fonksiyona doğru tipte değer göndermek programcının sorumluluğundadır. "Bilgisayar yanlış sonuç veriyor" diye düşündüğünüz çoğu durum küçük bir yazım veya tip hatasıdır.
- Algoritma analizinde temel işlemi seçin ve kaç kez yapıldığını toplam sembolüyle bulun. Bu, bütün kodu ayrıntılı saymaktan çok daha kısa yoldur.
- Bir değişkenin ulaştığı son değer ile algoritmanın karmaşıklığını karıştırmayın.
- Tutorial'ları ders içeriğine ek olarak kullanın. Bu bölüm kendi kendine öğrenmeyi gerektirir. Ezberle değil, soyut düşünerek öğrenin.

Kısa Tekrar Notları

- `return` tek değer döndürür. Birden fazla sonuç için adres gönderin (`g(&x, &y)`).
- `f('A')` ile `int` parametreye 65 gider. `char` parametreye 256 gönderilirse 0 olur.
- $T(n)$: temel işlemin n'e bağlı kaç kez yapıldığı. Katsayılar ve küçük terimler atılır.
- `for (i = 1; i <= n; i++)`: karşılaştırma $n+1$ kez, gövde n kez çalışır.
- $\sum_{i=1}^{n} 1 = n$, $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, $\sum_{i=1}^{n} c = cn$.
- `T = T + T` ($T_0 = 1$): 2^n. İç içe n ve n^2 döngü: n^3 iterasyon.
- $n = 10$ için sonuçlar: 10, 20, 55, 110, 1024, 1000, 5500, 50500.

Detaylı Açıklamalar

Derste analiz edilen döngüler

Derste tek tek değiştirilerek çalıştırılan döngüler, ders sonrasında tek bir programda toplanıp `gcc -Wall -std=c89 -pedantic` ile denendi:

```
T = 0; for (i = 1; i <= N; i++) T = T + 1;      /* T1 = n          -> 10    */
T = 0; for (i = 1; i <= N; i++) T = T + 2;      /* T2 = 2n         -> 20    */
T = 0; for (i = 1; i <= N; i++) T = T + i;      /* T3 = n(n+1)/2   -> 55    */
T = 0; for (i = 1; i <= N; i++) T = T + 2*i;    /* T4 = n(n+1)     -> 110   */
T = 1; for (i = 1; i <= N; i++) T = T + T;      /* T5 = 2^n        -> 1024  */
T = 0; for (i = 1; i <= N; i++)
           for (j = 1; j <= N*N; j++) T = T + 1; /* T6 = n^3        -> 1000  */
/* ayni ic ice dongude T = T + i:                  T7 = n^3(n+1)/2  -> 5500  */
/* ayni ic ice dongude T = T + j:                  T8 = n^3(n^2+1)/2 -> 50500 */
```

`N = 10` için çıktı: `T1=10 T2=20 T3=55 T4=110 T5=1024 T6=1000 T7=5500 T8=50500`. Formüllerle aynıdır.

Çift yönlü selection sort'un karşılaştırma sayısı

Ders sonrasında öğrencinin algoritmasının karşılaştırma sayısı hesaplandı. Öğrencinin anlattığı `else if` yapısında bir adımdaki karşılaştırma sayısı veriye göre 1 veya 2'dir. Her adımda 2 alınırsa üst sınır şudur:

$$\sum_{i=0}^{n/2-1} 2(n-2i-1) = \frac{n^2}{2}$$

Klasik selection sort $\frac{n(n-1)}{2}$ karşılaştırma yapar. $n = 1000$ için 499500'e karşı 500000 bulundu. Yani karşılaştırma sayısı neredeyse aynıdır. Kazanç, dış döngünün yarıya inmesinden ve döngü kontrollerinden gelir. Hocanın "iki kabul edersek başa baş gelebilir" uyarısı bu yüzden doğrudur. Gerçek farkı süre ölçümü gösterir.

Ders sonrası notlar

- **`f(5.40)` neden -1 veya -5 verdi?** Derste ilk çalıştırmada `f` ve `h` için prototip yoktu. Derleyici `conflicting types` ve `implicit declaration` uyarıları verdi. Prototip yokken `5.40` `double` olarak gönderilir, fonksiyon ise bir `int` okur. Bu tanımsız davranıştır (undefined behavior) ve ekrana anlamsız bir değer çıkar. Ders sonrasındaki denemede prototipsiz çağrı 1 yazdı. Prototip (`void f(int t);`) yazıldığında `5.40` önce `int`'e dönüştürülür (kesme, truncation) ve `f(5.40)` 5 yazar. Bu, fonksiyon prototiplerinin neden yazılması gerektiğinin iyi bir örneğidir.
- **`h(3)` ve ASCII:** Hoca `3`'ün ASCII kodunun (yaklaşık 51) karaktere dönüştürüldüğünü söyledi. Gönderilen değer `'3'` karakteri değil, 3 sayısıdır. `'3'` karakterinin kodu 51'dir, ama burada dönüşüm olmaz ve `t` 3 olarak kalır. `%c` ile 3 kodlu karakter basılır. Bu bir kontrol karakteridir ve Windows konsolu (kod sayfası 437) onu kalp işareti olarak gösterir.
- **ASCII ve `char` aralığı:** ASCII 0-127 arasıdır. 128-255 genişletilmiş kod sayfalarına aittir. `char` 8 bittir ve 256 farklı değer alabilir. `signed char` için en büyük değer 127, `unsigned char` için 255'tir. 256 ve 257'nin 0 ve 1 olması 256'ya göre mod almaya karşılık gelir. İşaretli tipe sığmayan bir değerin dönüşümü standartta derleyiciye bırakılmıştır (implementation-defined). GCC böyle davranır.
- **Java'da daraltma:** Java yalnızca örtük (implicit) daraltmaya izin vermez. `(char) uzunSayi` gibi açık bir cast ile dönüşüm yapılabilir.
- **Çift yönlü selection sort ve Quick Sort:** Quick Sort'un hızının asıl sebebi diziyi pivot etrafında iki parçaya bölüp parçaları ayrı ayrı sıralamasıdır (böl ve yönet). Bu sayede ortalama karmaşıklığı $n \log n$ olur. Tek adımda iki elemanın yer değiştirmesi bu karmaşıklığı açıklamaz. Yukarıda gösterildiği gibi çift yönlü selection sort hâlâ n^2'dir.
- **Ortada değişken tanımı ve `int a[n];`:** Değişkenleri bloğun başında tanımlama zorunluluğu C89 kuralıdır. C99'dan itibaren değişkenler kodun ortasında da tanımlanabilir ve `int a[n]` gibi değişken uzunluklu diziler (VLA) desteklenir. Bu ders ve laboratuvarlar C89 kurallarına göre değerlendirildiği için hocanın kuralına uyulmalıdır. VLA'lar yığında (stack) oluşturulur, bu yüzden büyük `n` değerlerinde program çöker.
- **Dizinin dışına erişim:** C dizi sınırlarını kontrol etmez. Program yalnızca erişilen adres işletim sisteminin o prosese vermediği bir bellek sayfasına düşerse çöker (segmentation fault veya Windows'ta access violation). Sınırın biraz dışına yazmak çoğu zaman hata vermeden başka değişkenlerin değerini bozar. Bu, çökmekten daha tehlikelidir.
- **Bütün diziyi -1 yapmak:** Döngü en açık yoldur. `int` dizisi için `memset(A, -1, sizeof(A));` (`<string.h>`) de çalışır, çünkü -1'in bütün baytları `0xFF`'tir. Bu hile yalnızca 0 ve -1 için geçerlidir. `memset(A, 5, ...)` elemanları 5 yapmaz.
- **Transpoz ve 90 derece döndürme:** Hoca, saat yönünün tersine 90 derece döndürmenin transpoz gibi düşünülebileceğini söyledi. Bu doğru değildir. Transpoz ana köşegene göre yansımadır. Saat yönünde 90 derece döndürme, transpoz alınıp her satırın ters çevrilmesidir. Saat yönünün tersine 90 derece döndürme, transpoz alınıp satırların sırasının ters çevrilmesidir. 1..9 matrisinin transpozu 1 4 7 / 2 5 8 / 3 6 9, saat yönünün tersine döndürülmüş hâli 3 6 9 / 2 5 8 / 1 4 7'dir.
- **`i++` sayısı:** Ayrıntılı sayımda `i++`, $N + 1$ olarak yazıldı. `i++` gövdeden sonra çalışır ve N kez yapılır (son değer $N+1$ olur). Karşılaştırma ise $N+1$ kez yapılır. Bu durumda toplam $4N + 3$'tür. Sonuç yine $T(N) \approx N$'dir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.